

Practical No. 1

Demonstrate to use of Version Control System (Git offline: on local machine with multiple user).

Create multiple users and usage with team leader role and coder roles.

Demonstrate code merge, fork (branching) and code diff amongst the users

Compare it with svn. (on answer sheet)

Procedure

Install Git

On Ubuntu:

```
sudo apt update  
sudo apt install git
```

Check:

```
git --version
```

Step 1: Create a Project Directory

```
mkdir gitdemo  
cd gitdemo  
git init
```

Step 2: Create a File

```
echo "Project Version 1" > app.txt
```

Step 3: Initial Commit by Team Leader

```
git add .  
git commit -m "Initial commit by Team Leader"
```

Check current branch:

```
git branch
```

Output:

```
* main
```

Step 4: Create Branch for Coder1

```
git branch coder1  
git checkout coder1
```

Add changes:

```
echo "Feature added by coder1" >> app.txt
```

Commit changes:

```
git add .  
git commit -m "Coder1 update"
```

Step 5: Create Branch for Coder2

Switch back to main:

```
git checkout main
```

Create branch:

```
git branch coder2  
git checkout coder2
```

Add changes:

```
echo "Feature added by coder2" >> app.txt
```

Commit changes:

```
git add .  
git commit -m "Coder2 update"
```

Step 6: Display All Branches

`git branch`

Sample Output:

```
main
coder1
* coder2
```

Step 7: Show Code Difference

Compare branches:

`git diff coder1 coder2`

This command displays the difference between coder1 and coder2 branches.

Step 8: Merge Branches

Switch to main branch:

`git checkout main`

Merge coder1:

`git merge coder1`

Merge coder2:

`git merge coder2`

If merge conflicts occur:

1. Open the file.
2. Resolve conflicts manually.
3. Save the file.

Then execute:

`git add .`
`git commit -m "Resolved merge conflict"`

Step 9: View Commit History

`git log --oneline --graph`

Sample Output:

- * Merge coder2
- * Coder2 update
- * Merge coder1
- * Coder1 update
- * Initial commit by Team Leader

Git	SVN (Subversion)
Distributed Version Control System (DVCS)	Centralized Version Control System (CVCS)
Every user has a complete copy of the repository	Repository is stored on a central server
Can work offline and commit changes locally	Requires connection to the central server for most operations
Faster branching and merging	Branching and merging are slower
Better performance for large projects	Comparatively slower
High fault tolerance since every clone is a backup	Central server failure can affect all users
Popular in modern software development	Used in some legacy projects and enterprises
Commits are local first, then pushed to remote	Commits are directly stored on the central server
Uses SHA hashes to identify commits	Uses sequential revision numbers (e.g., r1, r2, r3)
Widely used with GitHub, GitLab, and Gitea	Commonly used with SVN repositories and Apache Subversion